

Université Hassan I
École Nationale des Sciences Appliquées Berrechid

RAPPORT DE TRAVAUX PRATIQUES

Modélisation UML et Gestion de Versions

Module : Maquette et Ingénierie Numérique

Réalisé par : Saad Benhaimer

Encadrant : Pr. Hrimech Hamid

Année universitaire : 2025/2026

Table des matières

1	Création de la Scène	4
1.1	Création du Soleil	4
1.1.1	Paramètres du Soleil	4
1.1.2	Application de la Texture	5
1.2	Ajout des Planètes	6
1.3	Configuration du Skybox	7
2	Animation et Scripts	8
2.1	Principe d'Animation	8
2.2	Script d'Animation des Planètes	8
3	Contrôle de la Caméra	10
3.1	Script MouseAimCamera	10
3.2	Paramètres Importants	12

Résumé détaillé du TP – Système Solaire sous Unity 3D

Objectif général

Ce travail pratique a pour vocation la conception et l'animation d'une scène 3D interactive représentant le système solaire héliocentrique. L'apprenant doit modéliser le Soleil et les huit planètes, leur appliquer des textures réalistes, les animer par rotation propre et révolution orbitale, et naviguer dans la scène à l'aide d'une caméra contrôlée par la souris.

Compétences visées

À l'issue de ce TP, l'étudiant sera capable de :

- Manipuler les transformations spatiales (`Translate`, `Rotate`, `RotateAround`) dans un environnement 3D ;
- Importer et appliquer des textures sur des maillages sphériques ;
- Configurer un *Skybox* pour l'immersion environnementale ;
- Écrire des scripts C# pour l'animation temps réel via la méthode `Update()` ;
- Implémenter un système de caméra orbitale à la souris (*Mouse Aim*) ;
- Gérer l'arborescence des assets (dossiers `Textures`, `Materials`, `Audio`, `Scripts`).

Étapes de réalisation

Étape 1 – Importation des ressources

- Récupération des textures planétaires et solaires (<https://www.solarsystemscope.com/textures/>) ;
- Importation de l'asset *Starfield Skybox* pour l'arrière-plan spatial.

Étape 2 – Construction de la scène

- Création d'une sphère nommée « Soleil », positionnée à l'origine (0,0,0) et mise à l'échelle ;
- Création des huit planètes (Mercure, Vénus, Terre, Mars, Jupiter, Saturne, Uranus, Neptune) sous forme de sphères ;
- Organisation hiérarchique : création d'un dossier `Texture` contenant un sous-dossier `Material` pour l'application des textures ;
- Assignation du *Skybox* à la caméra via le *Rendering Settings*.

Étape 3 – Programmation des animations (C#) Deux comportements distincts sont implémentés pour chaque planète :

1. Rotation propre :

```
transform.Rotate(0, 35 * Time.deltaTime, 0);
```

2. Révolution orbitale autour du Soleil :

```
GameObject soleil = GameObject.Find("Soleil");
transform.RotateAround(soleil.transform.position,
                      -soleil.transform.up,
```

`70 * Time.deltaTime);`

Étape 4 – Contrôle de la caméra Développement d'un script `MouseAimCamera` permettant :

- La rotation orbitale de la caméra autour d'une cible (Soleil ou planète) selon l'axe horizontal de la souris (`Mouse X`) ;
- Le maintien d'un décalage (*offset*) constant entre la caméra et la cible ;
- L'orientation continue de la caméra vers la cible via `transform.LookAt()` ;
- L'extension du script pour intégrer le contrôle sur les axes **X et Y**.

Étape 5 – Enrichissement multimédia

- Création d'un dossier `Audio` ;
- Intégration d'une ambiance sonore spatiale (optionnel selon le fichier audio fourni).

Contraintes et points d'attention

- Le script de caméra doit être attaché à la `Main Camera` avec la cible (`target`) correctement référencée dans l'inspecteur ;
- La méthode `LateUpdate()` est privilégiée à `Update()` pour le contrôle caméra afin d'éviter les décalages de rendu ;
- Les vitesses de rotation et de révolution doivent être multipliées par `Time.deltaTime` pour garantir une animation indépendante du framerate.

Chapitre 1

Création de la Scène

1.1 Création du Soleil

Le Soleil constitue le point central de notre système solaire miniature, autour duquel gravitent tous les autres astres.

1.1.1 Paramètres du Soleil

Pour initialiser le Soleil, les étapes suivantes ont été appliquées dans l'interface d'Unity :

1. Insertion d'une sphère dans la scène (`GameObject > 3D Object > Sphere`).
2. Renommage de l'objet en « `Soleil` ».
3. Ajustement de la taille via la composante *Transform* (Scale réglé proportionnellement).
4. Positionnement rigoureux à l'origine absolue de la scène : $(0, 0, 0)$.

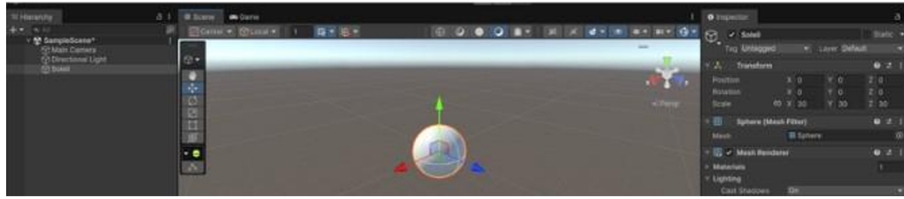


FIGURE 1.1 – Enter Caption

1.1.2 Application de la Texture

Afin de donner au Soleil son aspect lumineux et réaliste, une texture émissive spécifique a été importée et assignée à son *Material*. Les paramètres d'albedo et d'émission ont été configurés pour simuler le rayonnement de l'étoile.

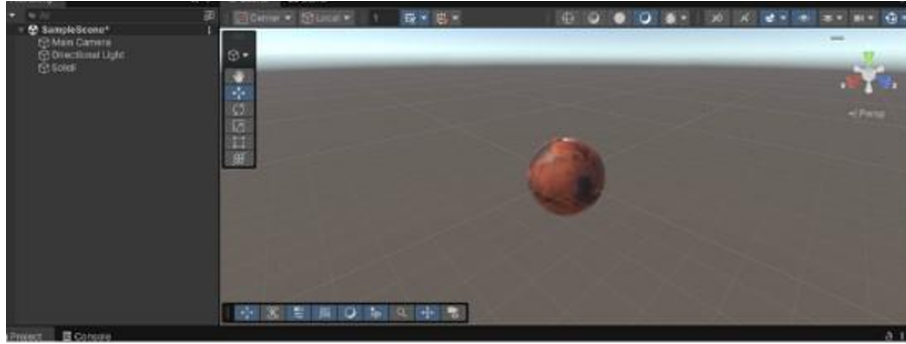


FIGURE 1.2 – Enter Caption

1.2 Ajout des Planètes

Suivant une logique similaire, les différentes planètes du système ont été générées sous forme de sphères à des distances croissantes de l'origine. Chacune possède des dimensions ajustées à sa nature ainsi qu'une texture dédiée mappée à son matériau standard pour les différencier visuellement.



FIGURE 1.3 – Enter Caption

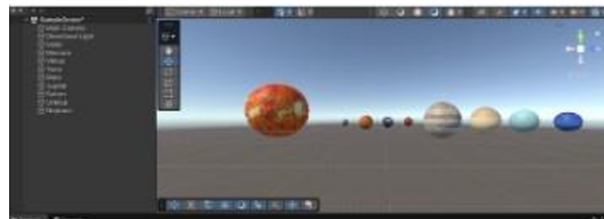


FIGURE 1.4 – Enter Caption

1.3 Configuration du Skybox

Pour immerger le système solaire dans un environnement spatial réaliste, le *Skybox* par défaut d'Unity a été remplacé. Un nouveau matériau de type *Skybox/6 Sided* a été créé, utilisant des textures de voûte céleste étoilée appliquées sur ses six faces internes.



FIGURE 1.5 – Enter Caption

Chapitre 2

Animation et Scripts

2.1 Principe d'Animation

Les mouvements au sein du système solaire reposent sur deux principes physiques distincts implémentés par programmation :

- **La Rotation (Self-Rotation)** : Le mouvement de l'astre sur lui-même autour de son axe vertical.
- **La Révolution (Orbit)** : Le déplacement orbital de la planète autour du Soleil central.

2.2 Script d'Animation des Planètes

Voici le script C# conçu pour gérer simultanément la rotation interne et la révolution orbitale de chaque planète :

```
1 using UnityEngine;
2
3 public class OrbitAndRotate : MonoBehaviour
4 {
5     [Header("Parametres de Revolution")]
6     public Transform centerObject; // Le Soleil
7     public float orbitSpeed = 20f;
8
9     [Header("Parametres de Rotation")]
10    public float rotationSpeed = 30f;
11
12    void Update()
13    {
14        // 1. Gestion de la Revolution (Orbite) autour du centre
15        if (centerObject != null)
16        {
17            transform.RotateAround(centerObject.position, Vector3.
18            up, orbitSpeed * Time.deltaTime);
19        }
20
21        // 2. Gestion de la Rotation de la planete sur elle-meme
22        transform.Rotate(Vector3.up, rotationSpeed * Time.deltaTime
23    );
24 }
```

```
22     }  
23 }
```

Listing 2.1 – Script d’animation OrbitAndRotate.cs

Chapitre 3

Contrôle de la Caméra

3.1 Script MouseAimCamera

Pour permettre à l'utilisateur de naviguer dynamiquement autour du système solaire en cours d'exécution, un script de contrôle de caméra basé sur les entrées de la souris a été implémenté.

```
1 using UnityEngine;
2
3 public class MouseAimCamera : MonoBehaviour
4 {
5     public Transform target; // L'objet à observer
6     public float distance = 10.0f;
7     public float xSpeed = 120.0f;
8     public float ySpeed = 120.0f;
9
10    public float yMinLimit = -20f;
11    public float yMaxLimit = 80f;
12
13    private float x = 0.0f;
14    private float y = 0.0f;
15
16    void Start()
17    {
18        Vector3 angles = transform.eulerAngles;
19        x = angles.y;
20        y = angles.x;
21
22        if (GetComponent<Rigidbody>())
23        {
24            GetComponent<Rigidbody>().freezeRotation = true;
25        }
26    }
27
28    void LateUpdate()
29    {
30        if (target && Input.GetMouseButton(1)) // Clic droit pour
orbiter
31        {
```

```

32         x += Input.GetAxis("Mouse X") * xSpeed * distance *
0.02f;
33         y -= Input.GetAxis("Mouse Y") * ySpeed * 0.02f;
34
35         y = ClampAngle(y, yMinLimit, yMaxLimit);
36
37         Quaternion rotation = Quaternion.Euler(y, x, 0);
38         Vector3 negDistance = new Vector3(0.0f, 0.0f, -distance
);
39         Vector3 position = rotation * negDistance + target.
position;
40
41         transform.rotation = rotation;
42         transform.position = position;
43     }
44 }
45
46 public static float ClampAngle(float angle, float min, float
max)
47 {
48     if (angle < -360F) angle += 360F;
49     if (angle > 360F) angle -= 360F;
50     return Mathf.Clamp(angle, min, max);
51 }
52 }

```

Listing 3.1 – Script de controle camera MouseAimCamera.cs

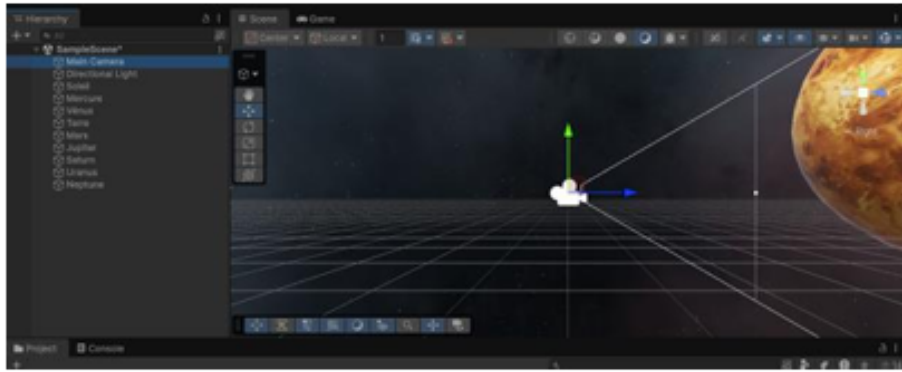


FIGURE 3.1 – Enter Caption

3.2 Paramètres Importants

- **target** : Référence le *Transform* de l'objet central à suivre des yeux (le Soleil).
- **distance** : Gère l'éloignement initial de la caméra par rapport à la cible.
- **xSpeed** / **ySpeed** : Coefficients de sensibilité définissant la réactivité face aux mouvements de la souris.

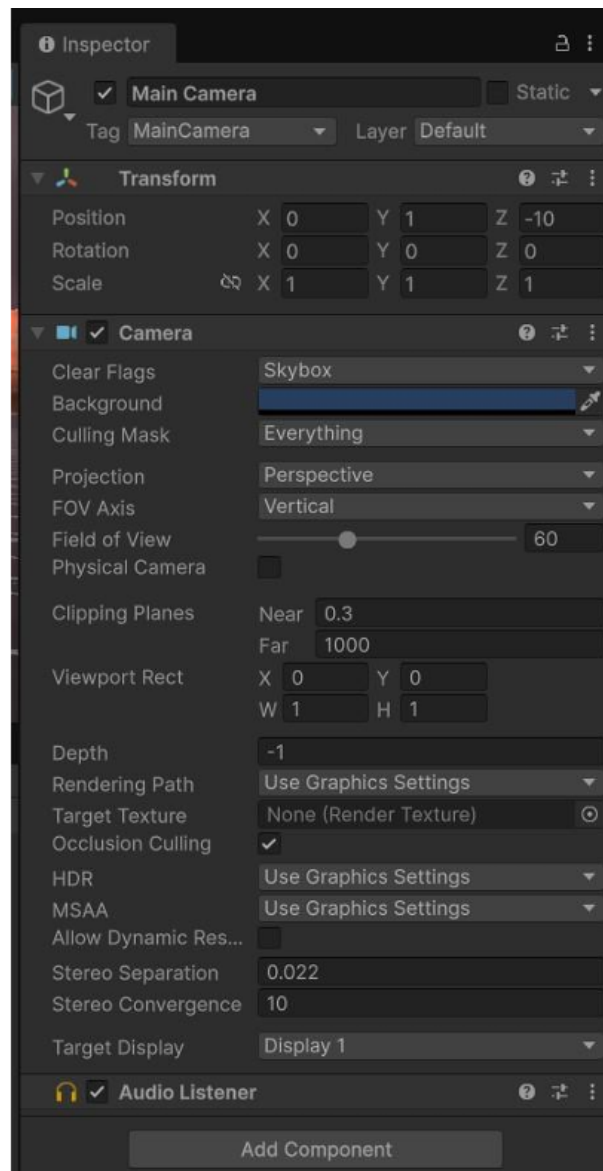


FIGURE 3.2 – Paramètres du script dans l'Inspector

Conclusion

Ce travail pratique a permis de modéliser une simulation interactive du système solaire sous Unity 3D, en combinant à la fois des aspects graphiques, algorithmiques et ergonomiques. À travers la création et l’animation des corps célestes, nous avons mis en œuvre les principes fondamentaux de transformation spatiale en trois dimensions — translation, rotation et orbite — tout en exploitant le rendu temps réel du moteur de jeu.

L’intégration de textures réalistes et d’un *Skybox* spatial a sensiblement amélioré l’immersion visuelle de la scène, tandis que la programmation en C# a offert un contrôle précis et paramétrable des trajectoires planétaires. Par ailleurs, l’implémentation d’une caméra contrôlée par la souris a enrichi l’interactivité du projet, permettant une exploration libre et intuitive de l’environnement modélisé.

Au-delà du résultat visuel, ce TP a constitué une initiation concrète au développement de scènes 3D temps réel, compétence transposable à de nombreux domaines de l’ingénierie des systèmes d’information : visualisation de données, simulation scientifique, réalité virtuelle ou encore conception de serious games. La maîtrise de ces outils et méthodologies ouvre ainsi la voie à des projets plus ambitieux alliant informatique graphique et interactivité utilisateur.